

LABORATORIO 17

Diez años de verdad

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio, y los tiempos en tu máquina van a ser otros.

Introducción

1 · El tema

Con una tabla de treinta mil documentos todo responde al instante, y eso esconde algo importante. No se nota la diferencia entre una tabla bien diseñada y una mal diseñada, porque las dos se sienten igual de rápidas.

En este laboratorio la bodega tiene diez millones de documentos y diez años de historia, y las mismas preguntas se hacen con el reloj a la vista. Vas a medir tres cosas. Leer solo el mes que se pide frente a revisar la bodega entera, contar sin abrir un solo papelito y lo que cuesta tener muchos papelitos chicos.

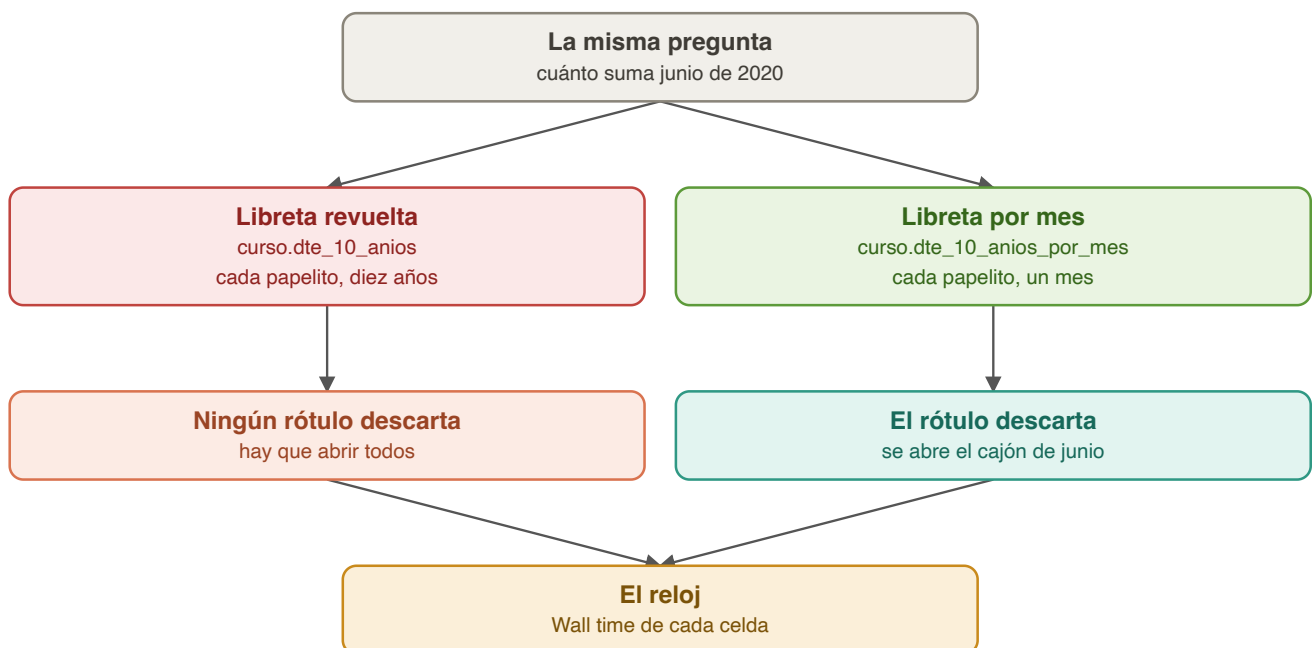
2 · El problema, en el almacén

La bodega de atrás tiene diez años de documentos. En una bodega están todos revueltos, y cada caja trae papeles de cualquier año. En otra hay un cajón por mes. Las dos tienen los mismos papeles y ocupan casi lo mismo. Cuando alguien pide el total de junio de 2020, en la bodega revuelta hay que abrir todas las cajas, porque cualquiera puede tener papeles de junio.

3 · El problema en términos técnicos

El motor decide qué archivos leer mirando lo que la tabla anota de cada uno. Si cada archivo trae filas de toda la historia, ninguno se puede descartar y la consulta lee la tabla entera aunque pida un mes. Y cuando una tabla se carga de a poco, un día a la vez, se llena de archivos chicos, y abrir cada archivo tiene un costo propio que se suma aunque traiga pocas filas.

4 · El diagrama



La misma pregunta (gris). Las dos libretas reciben exactamente la misma consulta, palabra por palabra.

Libreta revuelta (rojo). `curso.dte_10_anios` tiene los diez millones de documentos repartidos al azar. Cada papelito trae documentos de los diez años.

Libreta por mes (verde). `curso.dte_10_anios_por_mes` tiene los mismos documentos, con un cajón por mes. Cada papelito trae un solo mes.

Ningún rótulo descarta (coral). Cada papelito lleva un rótulo con el mínimo y el máximo de cada columna. En la revuelta todos los rótulos dicen de 2015 a 2024, así que junio de 2020 puede estar en cualquiera y hay que abrirlos todos.

El rótulo descarta (verde agua). En la libreta por mes, el rótulo de cada papelito dice un solo mes. El motor descarta los que no son junio de 2020 sin abrirlos.

El reloj (amarillo). La diferencia se mide con el tiempo que demora cada celda.

5 · La solución

En el almacén, se guardan los papeles en un cajón por mes y se le pega a cada caja una etiqueta con el primer y el último día que trae. Para buscar junio basta leer las etiquetas y abrir una caja. Y cuando llegan muchos papeles sueltos, de a uno por día, se pasan en limpio a una hoja grande de vez en cuando.

En la tecnología, el cajón es la partición por mes y la etiqueta es el rótulo, el mínimo y el máximo de cada columna que Iceberg anota de cada papelito. Pasar en limpio es `rewrite_data_files`, que junta muchos papelitos chicos en uno grande sin cambiar un dato. Y la regla que queda es medir, la misma pregunta con el reloj sobre los dos diseños, antes de decidir.

6 · Los pasos

- **Paso 0.** Calientas el motor con las dos libretas y cuentas los documentos leyendo solo la portada.
- **Paso 1.** Cuentas con `count(*)` la libreta entera.
- **Paso 2.** Sumas junio de 2020 en las dos libretas y comparas los tiempos.
- **Paso 3.** Mides cuánto pesa cada libreta en el almacenamiento.
- **Paso 4.** Mides una tabla con muchos papelitos chicos, la pasas en limpio y vuelves a medir.

Este laboratorio necesita dos preparaciones que no vienen hechas, porque demoran. Las dos se corren una vez desde la carpeta del repositorio. La primera crea las dos libretas de diez años, y demora entre veinte y cuarenta minutos. La segunda arma la tabla fragmentada del paso 4, y hay que volver a correrla cada vez que quieras repetir ese paso, porque el paso 4 la compacta.

```
bash
bin/crear-bodega.sh
bin/reponer-fragmentada.sh
```

Las dos libretas grandes están en el espacio `curso`, que tiene las tablas de referencia. En este laboratorio no se escribe en ellas, solo se leen y se mide. La tabla fragmentada está en tu espacio, `mi_espacio`.

Paso 0 · La bodega de diez años

Celda 0.1 · Calentar la libreta revuelta

La pregunta. ¿Cuántos documentos se emitieron el 5 de enero de 2015 en la libreta revuelta?

Por qué ahora. No se cronometra. La primera consulta enciende el motor, reserva memoria y se conecta al catálogo, y la primera vez que se toca una libreta hay que leer su portada. Ese trabajo se hace una sola vez. Si se midiera, el reloj mediría el arranque y no la pregunta.

En el almacén. Es abrir la puerta de la bodega y prender la luz antes de empezar a cronometrar.

La sentencia, parte por parte.

- `%%sql` le dice al cuaderno que la celda es SQL.
- `-- Celda 0.1` es un comentario, que el motor ignora.
- `SELECT count(*) AS documentos` cuenta las filas y le pone a la columna el nombre `documentos`.
- `FROM curso.dte_10_anios` es la libreta revuelta, en el espacio `curso`.
- `WHERE fecha_emision = DATE '2015-01-05'` se queda con un solo día. `DATE '...'` convierte el texto en una fecha.

```
sql
%%sql
-- Celda 0.1
SELECT count(*) AS documentos
FROM curso.dte_10_anios
WHERE fecha_emision = DATE '2015-01-05'
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:20:25 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
documentos
2234
1 fila.
```

Cómo se lee. Ese día se emitieron 2.234 documentos. Lo importante no es el número, es que el motor quedó despierto.

Las tres primeras líneas son avisos de Spark al arrancar, no errores.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar ese nivel. `sc` es el contexto de Spark. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop para este sistema operativo y usa la versión en Java. Funciona igual. `26/09/25 18:20:25` es la fecha y la hora del aviso, en UTC.

Celda 0.2 · Calentar la libreta por mes

La pregunta. ¿Cuántos documentos hay ese mismo día en la libreta por mes?

Por qué ahora. Para que las dos libretas queden despiertas y la comparación sea limpia.

En el almacén. Es prender la luz de la otra bodega.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta las filas.
- `FROM curso.dte_10_anios_por_mes` es la libreta con un cajón por mes.
- `WHERE fecha_emision = DATE '2015-01-05'` es el mismo día de la celda anterior.

```
sql
%%sql
-- Celda 0.2
SELECT count(*) AS documentos
FROM curso.dte_10_anios_por_mes
WHERE fecha_emision = DATE '2015-01-05'
```

Lo que sale en pantalla.

```
text
documentos
2234
1 fila.
```

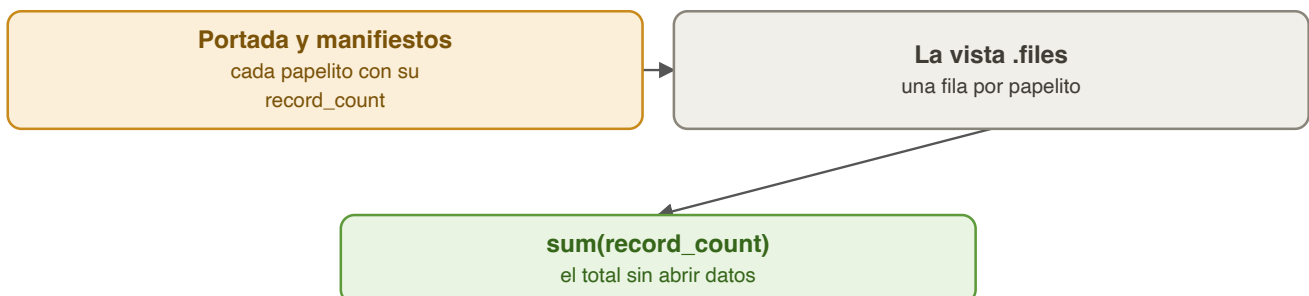
Cómo se lee. Los mismos 2.234 documentos. Las dos libretas tienen lo mismo ese día.

Celda 0.3 · Contar leyendo la portada, libreta revuelta

La pregunta. ¿Cuántos documentos tiene la libreta revuelta, sin abrir un solo papelito?

Por qué ahora. La portada de la libreta anota cuántas filas trae cada papelito. Sumar esa columna responde el total sin leer datos. Es la primera medición.

En el almacén. Es sumar lo que dicen las etiquetas de las cajas, sin abrir ninguna.



Portada y manifiestos (amarillo). La portada de la libreta y sus manifiestos, las listas de papelitos, guardan de cada papelito cuántas filas trae.

La vista .files (gris). Es la forma de consultar esa información. Cada fila es un papelito y `record_count` es cuántos documentos trae.

sum(record_count) (verde). Sumar esa columna da el total de documentos, y no se leyó ningún dato.

La sentencia, parte por parte.

- `%%time` es la primera línea. Es un cronómetro de Jupyter, y al terminar la celda imprime cuánto demoró. Va antes de `%%sql`.
- `SELECT sum(record_count) AS documentos` suma las filas que trae cada papelito.
- `FROM curso.dte_10_anios.files` es la vista de papelitos de la libreta revuelta. El nombre tiene tres partes, el espacio, la tabla y la vista.

```
sql
%%time
%%sql
-- Celda 0.3
SELECT sum(record_count) AS documentos
FROM curso.dte_10_anios.files
```

Lo que sale en pantalla.

```
text
documentos
10000000
1 fila.
CPU times: user 4.59 ms, sys: 3.07 ms, total: 7.67 ms
Wall time: 260 ms
```

Cómo se lee. Diez millones de documentos, y en 260 milisegundos, una cuarta parte de un segundo.

Las dos últimas líneas son del cronómetro.

- `CPU times` es cuánto trabajó el procesador para el proceso de Python del cuaderno. `user` es el tiempo en el programa mismo, `sys` el tiempo en el sistema operativo y `total` la suma, aquí 7.67 milisegundos. Es poco porque Python solo espera a Spark, que corre en otro proceso.
- `Wall time` es el tiempo de reloj de pared, el que habrías medido mirando el segundero. Es el que importa, y aquí es 260 ms.

Celda 0.4 · Contar leyendo la portada, libreta por mes

La pregunta. ¿Da lo mismo en la libreta por mes?

Por qué ahora. Para confirmar que las dos libretas tienen los mismos documentos y que leer la portada es rápido en las dos.

En el almacén. Es sumar las etiquetas de las cajas de la otra bodega.

La sentencia, parte por parte.

- `%%time` cronometra la celda.
- `SELECT sum(record_count) AS documentos` suma las filas de cada papelito.
- `FROM curso.dte_10_anios_por_mes.files` es la vista de papelitos de la libreta por mes.

```
sql
%%time
%%sql
-- Celda 0.4
SELECT sum(record_count) AS documentos
FROM curso.dte_10_anios_por_mes.files
```

Lo que sale en pantalla.

```
text
documentos
10000000
1 fila.
CPU times: user 8.49 ms, sys: 1.28 ms, total: 9.77 ms
Wall time: 136 ms
```

Cómo se lee. Los mismos diez millones, en 136 ms. Las dos libretas tienen lo mismo, y en las dos contar por la portada toma una fracción de segundo.

Paso 1 · Contar la bodega entera

Celda 1.1 · Contar con count(*)

La pregunta. ¿Cuánto demora contar diez millones de documentos como lo escribiría cualquiera?

Por qué ahora. Las celdas anteriores leyeron la portada a propósito. Esta hace la pregunta de la forma obvia, sin saber que la portada existe.

En el almacén. Es pedirle al bodeguero que cuente los papeles, sin decirle cómo.

La sentencia, parte por parte.

- `%%time` cronometra la celda.
- `SELECT count(*) AS documentos` cuenta todas las filas.
- `FROM curso.dte_10_anios` es la libreta revuelta completa, sin filtro.

```
sql
%%time
%%sql
-- Celda 1.1
SELECT count(*) AS documentos FROM curso.dte_10_anios
```

Lo que sale en pantalla.

```
text
documentos
10000000
1 fila.
CPU times: user 6.08 ms, sys: 1.85 ms, total: 7.93 ms
Wall time: 195 ms
```

Cómo se lee. El mismo número, en 195 ms. El motor tampoco contó los documentos. Reconoció que la pregunta era el total de filas, fue a la portada, sumó lo que estaba anotado papelito por papelito y respondió. Es lo mismo que hiciste a mano en las dos celdas anteriores.

Eso lo hace esta combinación de versiones, Iceberg 1.3.0 sobre Spark 3.3.4. Otra versión podría resolverlo distinto, y por eso se mide en vez de suponerlo.

Contar es gratis, pero leer no. La portada sabe cuántas filas hay en cada papelito, no qué dice cada fila. En cuanto la pregunta necesita los montos, hay que abrir papelitos.

Paso 2 · Junio de 2020

Celda 2.1 · Sumar junio en la libreta revuelta

La pregunta. ¿Cuánto suma junio de 2020 en la libreta revuelta, y cuánto demora?

Por qué ahora. Es la pregunta que de verdad se hace todos los meses, un mes entre ciento veinte, y necesita los montos.

En el almacén. Es pedir el total de junio de 2020 en la bodega donde todo está revuelto.

La sentencia, parte por parte.

- `%%time` cronometra la celda.
- `SELECT sum(monto_total) AS total_de_junio` suma el monto total de los documentos.
- `FROM curso.dte_10_anios` es la libreta revuelta.
- `WHERE fecha_emision BETWEEN '2020-06-01' AND '2020-06-30'` se queda con junio de 2020. `BETWEEN` incluye los dos extremos.

```
sql
%%time
%%sql
-- Celda 2.1
SELECT sum(monto_total) AS total_de_junio
FROM curso.dte_10_anios
WHERE fecha_emision BETWEEN '2020-06-01' AND '2020-06-30'
```

Lo que sale en pantalla.

```
text
total_de_junio
799678968699.81
1 fila.
CPU times: user 8.13 ms, sys: 1.96 ms, total: 10.1 ms
Wall time: 1.16 s
```

Cómo se lee. Junio de 2020 suma 799.678.968.699,81 pesos, y demoró 1.16 segundos. Anota ese tiempo, es el de la libreta revuelta.

Celda 2.2 · Sumar junio en la libreta por mes

La pregunta. ¿Cuánto demora la misma pregunta en la libreta por mes?

Por qué ahora. Es la comparación del laboratorio. La sentencia es la misma, palabra por palabra, y solo cambia la tabla.

En el almacén. Es pedir el mismo total en la bodega con un cajón por mes.

La sentencia, parte por parte.

- `%%time` cronometra la celda.
- `SELECT sum(monto_total) AS total_de_junio` suma los montos.
- `FROM curso.dte_10_anios_por_mes` es la libreta por mes.
- `WHERE fecha_emision BETWEEN '2020-06-01' AND '2020-06-30'` es el mismo filtro.

```
sql
%%time
%%sql
-- Celda 2.2
SELECT sum(monto_total) AS total_de_junio
FROM curso.dte_10_anios_por_mes
WHERE fecha_emision BETWEEN '2020-06-01' AND '2020-06-30'
```

Lo que sale en pantalla.

```
text
total_de_junio
799678968699.81
1 fila.
CPU times: user 4.86 ms, sys: 2.16 ms, total: 7.03 ms
Wall time: 171 ms
```

Cómo se lee. El mismo total, en 171 ms, casi siete veces menos que en la libreta revuelta.

La razón está en los rótulos. En la libreta por mes, el papelito de junio de 2020 tiene un rótulo que dice del 1 al 30 de junio de 2020, y los otros ciento diecinueve dicen otros meses. El motor descarta esos ciento diecinueve mirando solo los rótulos, que están en la portada, y abre uno. En la libreta revuelta cada papelito trae documentos de los diez años, así que todos los rótulos dicen de enero de 2015 a diciembre de 2024. Junio de 2020 cae dentro de todos, ninguno se descarta y hay que abrirllos todos y revisar fila por fila.

En tu máquina los tiempos van a ser otros. Lo que se repite es la proporción. La libreta por mes siempre responde varias veces más rápido.

Paso 3 · Cuánto pesa cada una

Celda 3.1 · El peso de la libreta revuelta

La pregunta. ¿Cuánto espacio ocupa la libreta revuelta en el almacenamiento?

Por qué ahora. Si una responde mucho más rápido, la pregunta natural es cuánto más espacio cuesta.

En el almacén. Es pesar la bodega entera, con cajas y todo.

La sentencia, parte por parte. No es SQL. Es una orden del sistema, y por eso empieza con `!`, que le dice al cuaderno que la mande a la terminal.

- `# Celda 3.1` es un comentario de Python.
- `hdfs dfs` es el cliente del sistema de archivos HDFS, donde viven las tablas.
- `-du` quiere decir disk usage, el espacio que ocupa una carpeta.
- `-s` pide un solo total para la carpeta, en vez de una línea por archivo.
- `-h` muestra el tamaño en unidades legibles, con una letra al final.
- `/warehouse/iceberg/curso.db/dte_10_anios` es la carpeta de la libreta revuelta.

```
bash
# Celda 3.1
!hdfs dfs -du -s -h /warehouse/iceberg/curso.db/dte_10_anios
```

Lo que sale en pantalla.

```
text
246.5 M  739.4 M  /warehouse/iceberg/curso.db/dte_10_anios
```

Cómo se lee. La libreta revuelta pesa 246.5 M, unos 258,5 MB.

Salen dos números y la carpeta.

- El primero, `246.5 M`, es lo que pesan los archivos. La `M` de HDFS es un mebibyte, 1.048.576 bytes, un poco más que un MB de un millón de bytes. Por eso 246.5 M son unos 258,5 MB.
- El segundo, `739.4 M`, es lo que el sistema de archivos cuenta con las copias de respaldo que pide al guardar. Aquí pide tres, y 739.4 es tres veces 246.5 menos el redondeo. Como este ambiente tiene un solo servidor de datos, la copia real es una. El número que importa es el primero.

Celda 3.2 · El peso de la libreta por mes

La pregunta. ¿Cuánto ocupa la libreta por mes?

Por qué ahora. Es la otra mitad de la comparación.

En el almacén. Es pesar la otra bodega.

La sentencia, parte por parte.

- `!hdfs dfs -du -s -h` es la misma orden de la celda anterior.
- `/warehouse/iceberg/curso.db/dte_10_anios_por_mes` es la carpeta de la libreta por mes.

```
bash
# Celda 3.2
!hdfs dfs -du -s -h /warehouse/iceberg/curso.db/dte_10_anios_por_mes
```

Lo que sale en pantalla.

```
text
227.0 M   681.1 M   /warehouse/iceberg/curso.db/dte_10_anios_por_mes
```

Cómo se lee. La libreta por mes pesa 227.0 M, unos 238 MB. Es un poco menos que la revuelta.

Ordenar no cuesta espacio. Son los mismos diez millones de documentos y la misma cantidad de papelitos, 120 en cada una. Pesa algo menos porque las filas parecidas quedan juntas y se comprimen mejor, y los documentos del mismo mes se parecen más entre sí que documentos tomados al azar de diez años.

Paso 4 · Muchos papelitos chicos

Celda 4.1 · Cuántos papelitos tiene la tabla fragmentada

La pregunta. ¿En cuántos papelitos está guardada la tabla fragmentada?

Por qué ahora. Es el punto de partida de la tercera medición. Esta tabla tiene un año de documentos, un millón, cargado con una escritura por día, como lo haría una ingesta diaria. Son 366 páginas, una por cada día de 2024.

En el almacén. Es contar los papeles sueltos que se fueron acumulando, uno o varios por día, durante un año.

La sentencia, parte por parte.

- `SELECT count(*) AS papelitos` cuenta las filas de `.files`, es decir, los papelitos.
- `sum(record_count) AS documentos` suma los documentos que trae cada uno.
- `sum(file_size_in_bytes) AS bytes` suma lo que pesa cada papelito.
- `FROM mi_espacio.dte_un_anio_fragmentado.files` es la vista de papelitos de la tabla fragmentada, en tu espacio.

```
sql
%%sql
-- Celda 4.1
SELECT count(*)           AS papelitos,
       sum(record_count)   AS documentos,
       sum(file_size_in_bytes) AS bytes
FROM mi_espacio.dte_un_anio_fragmentado.files
```

Lo que sale en pantalla.

```
text
papelitos documentos bytes
1464      1000000 33476440
1 fila.
```

Cómo se lee. 1464 papelitos para un millón de documentos, unos setecientos documentos por papelito. Entre todos pesan 33476440 bytes, 33,48 MB. Son papelitos diminutos.

Celda 4.2 · Calentar la tabla fragmentada

La pregunta. ¿Cuántos documentos hay el 15 de marzo de 2024?

Por qué ahora. No se cronometra, por la misma razón que en el paso 0. Es la primera vez que se leen datos de esta tabla, y esa primera lectura paga el arranque. Sin esta celda el tiempo de antes saldría inflado.

En el almacén. Es prender la luz del cuarto donde están los papeles sueltos.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta filas.
- `FROM mi_espacio.dte_un_anio_fragmentado` es la tabla fragmentada.
- `WHERE fecha_emision = DATE '2024-03-15'` se queda con un día.

```
sql
%%sql
-- Celda 4.2
SELECT count(*) AS documentos
FROM mi_espacio.dte_un_anio_fragmentado
WHERE fecha_emision = DATE '2024-03-15'
```

Lo que sale en pantalla.

```
text
documentos
2423
1 fila.
```

Cómo se lee. 2423 documentos ese día. El número no importa, la tabla quedó despierta.

Celda 4.3 · El año entero, antes de pasar en limpio

La pregunta. ¿Cuánto demora sumar el año con mil cuatrocientos papelitos chicos?

Por qué ahora. Es el número de antes.

En el almacén. Es sumar un año de papeles sueltos, abriendo cada uno.

La sentencia, parte por parte.

- `%%time` cronometra la celda.
- `SELECT sum(monto_total) AS total_del_anio` suma los montos de todo el año.
- `FROM mi_espacio.dte_un_anio_fragmentado` es la tabla fragmentada, sin filtro.

```
sql
%%time
%%sql
-- Celda 4.3
SELECT sum(monto_total) AS total_del_anio
FROM mi_espacio.dte_un_anio_fragmentado
```

Lo que sale en pantalla.

```
text
total_del_anio
9646638010265.48
1 fila.
CPU times: user 5.99 ms, sys: 2.32 ms, total: 8.31 ms
Wall time: 3.28 s
```

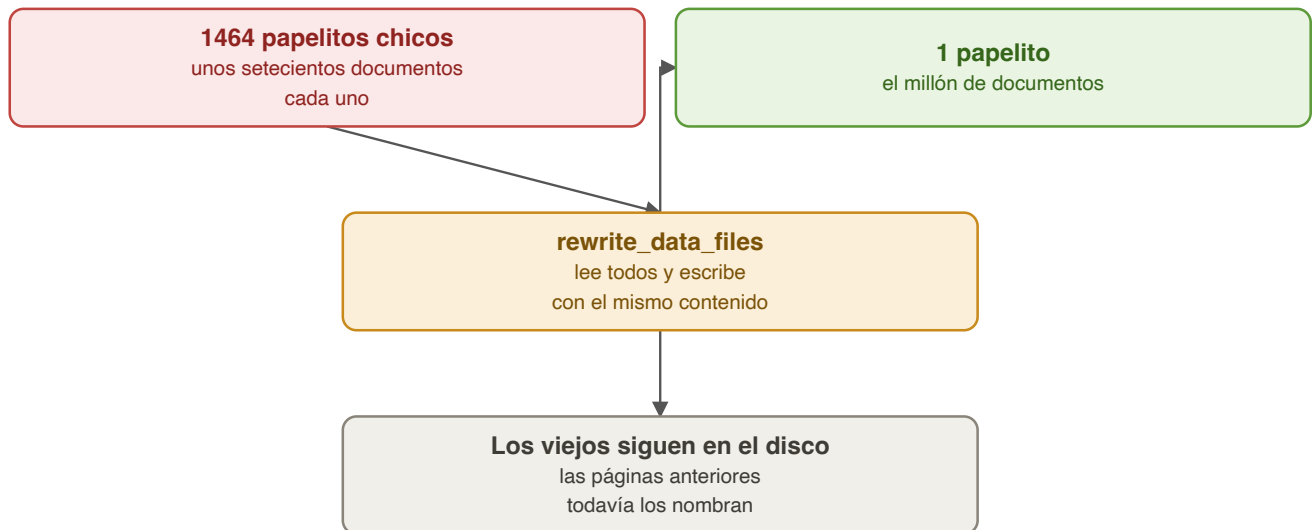
Cómo se lee. El año suma 9.646.638.010.265,48 pesos y demoró 3.28 segundos. Es el costo de abrir mil cuatrocientos veces.

Celda 4.4 · Pasar en limpio

La pregunta. ¿Cómo se juntan los papelitos chicos en uno grande?

Por qué ahora. Es la cura del problema.

En el almacén. Es pasar en limpio todos los papeles sueltos a una hoja grande, sin cambiar lo que dicen.



1464 papelitos chicos (rojo). Son los papelitos de la ingesta diaria, uno o varios por día.

rewrite_data_files (amarillo). Es el procedimiento de Iceberg que lee los papelitos chicos y escribe papelitos grandes con el mismo contenido. No cambia ni un dato.

1 papelito (verde). Con un millón de documentos, todo cabe en uno solo. Iceberg apunta a papelitos de hasta 536.870.912 bytes, unos 537 MB, y este queda muy por debajo.

Los viejos siguen en el disco (gris). Pasar en limpio no borra nada. Los papelitos viejos siguen ahí mientras las páginas anteriores los nombren.

La sentencia, parte por parte.

- `%%time` cronometra la celda.
- `CALL` ejecuta un procedimiento.
- `spark_catalog.system.rewrite_data_files` es el procedimiento de Iceberg que pasa en limpio, en el catálogo `spark_catalog`.
- `('mi_espacio.dte_un_anio_fragmentado')` es la tabla, con su espacio, entre comillas.

```
sql
%%time
%%sql
-- Celda 4.4
CALL spark_catalog.system.rewrite_data_files('mi_espacio.dte_un_anio_fragmentado')
```

Lo que sale en pantalla.

```
text
rewritten_data_files_count  added_data_files_count  rewritten_bytes_count
1464                        1                        33476440
1 fila.
CPU times: user 11.7 ms, sys: 260 µs, total: 11.9 ms
Wall time: 11 s
```

Cómo se lee. Leyó 1464 papelitos y escribió uno, en 11 segundos.

- `rewritten_data_files_count` es cuántos papelitos leyó y reemplazó, 1464.
- `added_data_files_count` es cuántos escribió, 1.
- `rewritten_bytes_count` es cuántos bytes leyó, 33476440, los mismos de la celda 4.1.

En el cronómetro, `260 µs` son 260 microsegundos, millonésimas de segundo.

Celda 4.5 · Cuántos papelitos quedaron

La pregunta. ¿Cómo quedó la tabla después de pasar en limpio?

Por qué ahora. Para ver el resultado con la misma consulta de la celda 4.1.

En el almacén. Es contar las hojas después de pasar en limpio.

La sentencia, parte por parte.

- `SELECT count(*) AS papelitos` cuenta papelitos.
- `sum(record_count) AS documentos` suma documentos.
- `sum(file_size_in_bytes) AS bytes` suma bytes.
- `FROM mi_espacio.dte_un_anio_fragmentado.files` es la vista de papelitos vigentes.

```
sql
%%sql
-- Celda 4.5
SELECT count(*)           AS papelitos,
       sum(record_count)   AS documentos,
       sum(file_size_in_bytes) AS bytes
FROM mi_espacio.dte_un_anio_fragmentado.files
```

Lo que sale en pantalla.

```
text
papelitos  documentos  bytes
1           1000000    22751985
1 fila.
```

Cómo se lee. Un solo papelito con el millón de documentos, y pesa 22751985 bytes, 22,75 MB. Es menos que los 33476440 bytes de antes, porque juntos se comprimen mejor y hay una sola cabecera en vez de mil cuatrocientas.

Celda 4.6 · El año entero, después de pasar en limpio

La pregunta. ¿Cuánto demora ahora la misma suma?

Por qué ahora. Es el número de después.

En el almacén. Es sumar el año leyendo una sola hoja.

La sentencia, parte por parte.

- `%%time` cronometra la celda.
- `SELECT sum(monto_total) AS total_del_anio` suma los montos del año.
- `FROM mi_espacio.dte_un_anio_fragmentado` es la misma tabla, ya compactada.

```
sql
%%time
%%sql
-- Celda 4.6
SELECT sum(monto_total) AS total_del_anio
FROM mi_espacio.dte_un_anio_fragmentado
```

Lo que sale en pantalla.

```
text
total_del_anio
9646638010265.48
1 fila.
CPU times: user 5.82 ms, sys: 1.92 ms, total: 7.74 ms
Wall time: 197 ms
```

Cómo se lee. El mismo total, en 197 ms. Antes demoró 3.28 segundos, así que ahora es unas dieciséis veces más rápido. El dato no cambió, cambió cuántas veces hubo que abrir un papelito.

Celda 4.7 · Lo que sigue en el disco

La pregunta. Si ahora hay un solo papelito, ¿bajó el espacio en el disco?

Por qué ahora. Es lo que casi nadie espera. Parece que el disco debería haber bajado, y no.

En el almacén. Es darse cuenta de que los papeles sueltos siguen en el cuarto, al lado de la hoja limpia.

La sentencia, parte por parte.

- `(SELECT count(*) FROM ... .files) AS papelitos_vigentes` cuenta los papelitos que nombra la página de hoy.
- `(SELECT sum(file_size_in_bytes) FROM ... .files) AS bytes_vigentes` suma lo que pesan.
- `(SELECT count(DISTINCT file_path) FROM ... .all_files) AS papelitos_en_disco` cuenta los papelitos que nombra cualquier página, la de hoy y las anteriores. `.all_files` es esa vista. `DISTINCT file_path` cuenta cada archivo una vez, porque un mismo papelito aparece una vez por cada manifiesto que lo nombra.
- `(SELECT sum(bytes) FROM (SELECT DISTINCT file_path, file_size_in_bytes AS bytes FROM ... .all_files)) AS bytes_en_disco` suma lo que pesan esos papelitos, cada uno una sola vez.

```

sql
%%sql
-- Celda 4.7
SELECT (SELECT count(*)          FROM mi_espacio.dte_un_anio_fragmentado.files) AS
papelitos_vigentes,
      (SELECT sum(file_size_in_bytes)  FROM mi_espacio.dte_un_anio_fragmentado.files) AS
bytes_vigentes,
      (SELECT count(DISTINCT file_path) FROM mi_espacio.dte_un_anio_fragmentado.all_files) AS
papelitos_en_disco,
      (SELECT sum(bytes) FROM (SELECT DISTINCT file_path, file_size_in_bytes AS bytes
                              FROM mi_espacio.dte_un_anio_fragmentado.all_files)) AS
bytes_en_disco

```

Lo que sale en pantalla.

```

text
papelitos_vigentes  bytes_vigentes  papelitos_en_disco  bytes_en_disco
1                  22751985      1465                56228425
1 fila.

```

Cómo se lee. El disco no bajó, subió. La tabla usa 1 papelito de 22751985 bytes, pero en el disco hay 1465 papelitos que pesan 56228425 bytes, 56,23 MB. Son los 1464 viejos más el nuevo, casi el doble de antes de compactar.

No es una falla. Las páginas viejas de la libreta todavía nombran esos papelitos, y mientras existan se puede pedir la tabla como estaba ayer. El espacio se libera cuando se botan esas páginas con `expire_snapshots`, y eso es una decisión aparte. Cuánta historia se conserva no lo decide quien compacta, lo decide la exigencia de auditoría de la institución.

Preguntas frecuentes

¿Por qué mis tiempos son distintos a los de la guía?

Porque dependen de tu máquina, de cuánta memoria le diste a Docker y de qué más esté haciendo el computador. Lo que se repite es la proporción. La libreta por mes siempre es varias veces más rápida para un mes, y la tabla compactada siempre es más rápida que la fragmentada.

¿Por qué `count(*)` no demoró nada si son diez millones?

Porque en esta versión el motor responde el total de filas con la portada, sumando el `record_count` de cada papelito, sin abrir ninguno. Si la consulta tuviera un filtro por una columna o pidiera montos, tendría que leer.

¿Qué es la M que muestra hdfs?

Es un mebibyte, 1.048.576 bytes. El MB que usa esta guía es de un millón de bytes, así que un número en M es un poco más grande en MB. 246.5 M son unos 258,5 MB.

¿Por qué el segundo número de hdfs es el triple?

Porque el sistema de archivos pide tres copias de cada archivo por si falla un servidor, y ese número cuenta las tres. En este ambiente hay un solo servidor de datos, así que en la práctica hay una copia. El número que dice cuánto pesan los datos es el primero.

¿Conviene particionar siempre por mes?

No. Particionar tiene su costo, porque crea más papelitos y cada carga chica deja los suyos. La regla es medir la pregunta que de verdad se hace, con el reloj, sobre los dos diseños antes de decidir.

¿Por qué la compactación dejó un solo papelito y no varios?

Porque Iceberg apunta a papelitos de hasta unos 537 MB, y el año entero comprimido pesa unos 22,75 MB. Con una tabla más grande quedarían varios papelitos, cada uno cerca de ese tamaño.

¿Cómo libero el espacio de los papelitos viejos?

Con `expire_snapshots`, que bota las páginas anteriores a una fecha de corte y con ellas los papelitos que solo esas páginas usaban. Después de eso ya no se puede pedir la tabla como estaba en esas páginas. Por eso la fecha de corte se decide antes, según cuánta historia exige conservar la auditoría.

¿Qué pasa si repito el paso 4 sin reponer la tabla?

La tabla ya está compactada, así que la celda 4.1 va a mostrar pocos papelitos y la compactación no tendrá nada que juntar. Para repetirlo, corre antes `bin/reponer-fragmentada.sh`, que demora unos diez minutos.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Con treinta mil filas cualquier diseño parece bueno. Con diez millones, la diferencia entre un diseño y otro es la diferencia entre un reporte que sale y uno que no. Y la única forma de saber de qué lado estás es medir la misma pregunta sobre los dos diseños, con el reloj, antes de decidir.

En el almacén. Las dos bodegas tenían los mismos papeles y pesaban casi lo mismo. En la que tenía un cajón por mes, buscar junio fue abrir una caja. Y el cuarto de los papeles sueltos se volvió rápido al pasarlos en limpio, aunque los papeles viejos siguieron ahí hasta que alguien decidiera botarlos.



Contar (amarillo). Contar documentos se responde con la portada, en las dos libretas y en décimas de segundo.

Un mes (verde). Sumar un mes es donde las libretas se separan. El rótulo deja descartar los cajones de otros meses en la libreta por mes, y en la revuelta no.

Papelitos chicos (azul). Muchos papelitos chicos cuestan caro. Pasarlos en limpio los deja en uno, y el espacio no baja hasta que se botan las páginas viejas.

Medir antes de decidir (gris). La regla que queda es la misma pregunta, con el reloj, sobre los dos diseños.

Los números de la solución ejecutada.

Medición	Libreta revuelta o antes	Libreta por mes o después
Contar por la portada	260 ms	136 ms
Contar con <code>count(*)</code>	195 ms	no se midió
Sumar junio de 2020	1.16 s	171 ms
Peso en HDFS	246.5 M	227.0 M
Papelitos de la tabla fragmentada	1464	1
Sumar el año de la tabla fragmentada	3.28 s	197 ms
Papelitos en el disco después de compactar	1465	

Las tablas con las que vas a trabajar no tienen treinta mil filas.